

Roteiro de vídeo + tutorial do pipeline (Jenkins + Docker)

Daniel Trindade — Gerência de Configuração · UFAM

Este documento é um **passo a passo para configurar o pipeline** e um **roteiro de gravação** dos vídeos de cada cenário. A ideia é abrir o OBS, seguir as cenas na ordem e narrar os textos sugeridos. Cada cenário roda o **build em um container Docker** e os **testes em outro container isolado** (ver [arquitetura-docker.md](#)).

Sugestão de hospedagem: grave com OBS, suba no YouTube (não listado) ou Google Drive e cole o link no slide correspondente da apresentação.

Parte 0 — Preparação (faça uma vez, antes de gravar)

0.1 Checklist da máquina

- ☐ **Docker Desktop** aberto e no modo **Linux containers** (ícone na bandeja).
- ☐ **Docker CLI** acessível ao Jenkins (rodar `docker --version` na máquina do Jenkins). O plugin **Docker Pipeline** **não** é necessário — subimos os containers com `docker run`. Só o plugin **Git** é preciso, para o checkout.
- ☐ Jenkins acessível em `http://localhost:8080`.
- ☐ (Opcional, mas ajuda no vídeo) baixar a imagem antes para o build não gastar tempo baixando durante a gravação: `bash docker pull node:22-alpine`

0.2 Layout de tela recomendado (OBS)

Deixe **duas janelas visíveis** ao mesmo tempo:

1. O **navegador no Jenkins** (ocupa a maior parte da tela).
2. O **Docker Desktop na aba "Containers"** num canto — assim dá para ver os containers de build e de teste **subindo e sumindo** durante o pipeline.

0.3 Montar o job (uma vez só)

1. **New Item** → nome `conversao-temperatura-docker` → tipo **Pipeline** → OK.
2. Em **Pipeline** → **Definition**, escolha **"Pipeline script from SCM"**.
3. **SCM: Git**.
 - **Repository URL:** `https://github.com/DanielTrindade/jenkins-trabalho.git`
 - **Credentials:** *(none)* — o repositório é público.
 - **Branch Specifier:** `*/main` (vamos trocar isso por cenário).
4. **Script Path:** `Jenkinsfile`.
5. **Save**.

Como alternar de cenário: cada cenário está em uma branch própria. Para trocar, entre em **Configure**, mude o **Branch Specifier** para a branch do cenário, **Save** e rode **Build Now**. (Alternativa: criar um job separado por branch — mas trocar o Branch Specifier é mais rápido para gravar.)

Cenário	Branch (Branch Specifier)
1 — sucesso	*/conversion-cenario-build-sucesso
2 — falha de build	*/conversion-cenario-falha-sintaxe
3 — instável	*/conversion-cenario-testes-falhando
4 — nightly	*/conversion-cenario-build-agendado

0.4 O que comprova "dois containers isolados"

Em **todos** os vídeos, ao abrir o **Console Output**, aponte para:

- as linhas `docker run ... node:22-alpine` (o Jenkins subindo cada container);
- no stage de build, o bloco `=== BUILD ===` seguido do **hostname do container** (um ID tipo `ef644d17ddd8`);
- no stage de teste, o bloco `=== TESTE ===` seguido de **outro hostname** (ID diferente) — provando que é um segundo container, isolado do primeiro.

Cenário 1 — "Tudo certo!" (build + testes com sucesso)

Branch: conversion-cenario-build-sucesso · **Resultado:** SUCCESS · ~2 min

#	Ação na tela	Narração sugerida
1	Mostrar o job em Configure , com o Branch Specifier em <code>*/conversion-cenario-build-sucesso</code> . Salvar.	"Esse é o nosso pipeline. Ele está apontando para a branch do cenário de sucesso. Repare que o build e o teste vão rodar em containers Docker."
2	Clicar em Build Now .	"Vou disparar o job manualmente."
3	Apontar para o Docker Desktop : o container de build aparece e some; depois o de teste.	"Olha aqui no Docker: subiu o primeiro container, o de build. Quando ele termina, sobe um segundo container, o de teste."
4	Abrir Stage View /	"No pipeline, primeiro o

#	Ação na tela	Narração sugerida
	Console Output. Mostrar o stage Build verde e depois Test verde.	estágio de build passou; em seguida, o de teste."
5	No Console Output , destacar os dois hostname diferentes e o node:22-alpine.	"Aqui está a prova do isolamento: o build rodou neste container, e o teste rodou neste outro — hostnames diferentes."
6	Mostrar o resultado SUCCESS (bola azul/verde) e o Test Result com 9 testes passando.	"Resultado: SUCCESS. Os 9 casos de teste passaram. Build verde."

Resultado esperado: build e testes em containers separados, ambos com sucesso, JUnit publicado, status **SUCCESS**.

Cenário 2 — "Deu ruim!" (falha na compilação)

Branch: conversion-cenario-falha-sintaxe · **Resultado:** FAILURE · ~2 min

O que muda nessa branch: em `src/app.js` há um erro de sintaxe proposital — `const { pathname = requestUrl; (a desestruturação ficou sem fechar)`. Isso faz o `npm run check` (nossa "compilação") falhar **dentro do container de build**.

#	Ação na tela	Narração sugerida
1	Em Configure , trocar o Branch Specifier para */conversion- cenario-falha- sintaxe. Salvar.	"Agora aponto o job para a branch com um erro de sintaxe no código."
2	(Opcional) Mostrar rapidamente o trecho quebrado em <code>src/app.js</code> no GitHub.	"Esse é o erro: a desestruturação ficou sem fechar a chave."
3	Clicar em Build Now .	"Disparo o job manualmente."
4	No Console Output , mostrar o stage Build falhando em <code>npm run</code> <code>check</code> com	"O container de build subiu, rodou o check e quebrou na compilação — olha o <code>SyntaxError</code> ."

#	Ação na tela	Narração sugerida
	SyntaxError.	
5	Destacar que o stage Test nem aparece / foi pulado (Stage "Test" skipped due to earlier failure(s)).	"Repare: o estágio de teste nem chegou a rodar. Como o build falhou, o Jenkins pula o teste — o segundo container nem é criado."
6	Mostrar o resultado FAILURE (bola vermelha).	"Resultado: FAILURE. A falha foi na etapa de build, exatamente como esperado."

Resultado esperado: build falha na compilação **dentro do container de build**; o container de teste não é criado; status **FAILURE**.

Cenário 3 — "Tá instável!" (build ok, teste falha → UNSTABLE)

Branch: conversion-cenario-testes-falhando · **Resultado:** UNSTABLE · ~2 min

O que muda nessa branch: em `src/conversion-service.js` a fórmula virou (`value - 32) * (4 / 9)` (deveria ser `5 / 9`). A sintaxe continua válida (build passa), mas o caso de teste `fahrenheitToCelsius(32) === 0` falha.

#	Ação na tela	Narração sugerida
1	Em Configure , trocar o Branch Specifier para <code>*/conversion-cenario-testes-falhando</code> . Salvar.	"Agora a branch onde o build compila, mas a regra de negócio está errada."
2	(Opcional) Mostrar a fórmula alterada em <code>conversion-service.js</code> no GitHub.	"Troquei o 5/9 por 4/9 de propósito — a conversão fica errada."
3	Clicar em Build Now .	"Disparo o job."
4	No Console Output , mostrar o stage Build verde (check passou).	"O container de build passou normalmente: a sintaxe está válida."
5	Mostrar o stage Test rodando no segundo container e um caso falhando .	"Já no container de teste, um caso falha — a conversão não bate com o valor esperado."
6	Apontar o status	"Resultado:

#	Ação na tela	Narração sugerida
	UNSTABLE (bola amarela) e abrir o Test Result mostrando o teste vermelho.	UNSTABLE, não FAILURE. O <code>catchError</code> capturou a falha e marcou o build como instável, mas publicou o relatório JUnit."

Resultado esperado: build verde no primeiro container; teste falha no segundo; JUnit publicado; status **UNSTABLE** (amarelo).

Cenário 4 — "O lendário nightly" (build agendado, sem intervenção)

Branch: conversion-cenario-build-agendado · **Resultado:** SUCCESS · ~2-3 min

O que muda nessa branch: o Jenkinsfile usa `cron('* * * * *')`, ou seja, dispara **a cada minuto**. É só para a demonstração — em produção voltaria para algo como `H 8 * * 1-5`.

#	Ação na tela	Narração sugerida
1	Em Configure , trocar o Branch Specifier para <code>*/conversion-cenario-build-agendado</code> . Save .	"Última branch: aqui o pipeline tem um agendamento via cron."
2	Mostrar o trecho <code>triggers { cron('* * * * *') }</code> no Jenkinsfile.	"Esse gatilho faz o Jenkins rodar o job sozinho, a cada minuto, só para a demonstração."
3	Não clicar em Build Now . Ficar parado na tela do job, com o relógio à vista.	"Repare que eu não vou clicar em nada. Vou apenas esperar o horário chegar."
4	Quando o build iniciar sozinho, apontar para o novo build na fila/histórico.	"Pronto: o build começou sozinho, sem eu tocar em nada."
5	Abrir o build e mostrar "Started by timer" no topo do Console Output.	"E aqui está a prova: 'Started by timer'. Foi o agendador que disparou, não eu."
6	Mostrar build e teste passando nos dois	"Mesmo fluxo de sempre: container de

#	Ação na tela	Narração sugerida
	containers e o status SUCCESS .	build, depois container de teste. Resultado SUCCESS, automático."

Resultado esperado: o job inicia **automaticamente** ("Started by timer"); build e testes passam nos dois containers; status **SUCCESS**.

Dica: depois de gravar, lembre de voltar o Branch Specifier para */main ou desabilitar o job, senão ele continua disparando a cada minuto.

Bônus — Cobertura de código

Qualquer branch de sucesso · ~1 min

A cobertura já roda em todo build (o `npm run test:coverage` usa a flag nativa `--experimental-test-coverage` e gera os relatórios). Para destacar no vídeo:

#	Ação na tela	Narração sugerida
1	No Console Output do build de sucesso, rolar até a tabela de cobertura do Node.	"Além de passar nos testes, a execução também mede a cobertura de código, linha a linha."
2	Na página do build, abrir os artefatos arquivados e mostrar <code>reports/lcov.info</code> e <code>reports/junit.xml</code> .	"O Jenkins arquiva dois relatórios: o JUnit, com o resultado dos testes, e o lcov, com a cobertura."
3	(Opcional) Mostrar o gráfico de tendência de testes do job.	"Com o JUnit publicado, o Jenkins ainda monta esse gráfico de tendência dos testes ao longo dos builds."

Resultado esperado: build e testes com sucesso + relatórios de cobertura (`lcov.info`) e de testes (`junit.xml`) visíveis/arquivados.

Resumo rápido (cola para gravar)

Cenário	Branch	Dispara	Resultado	Prova-chave no vídeo
1	...build-sucesso	Build Now	SUCCESS	2 hostnames diferentes; 9

Cenário	Branch	Dispara	Resultado	Prova-chave no vídeo
2	...falha- sintaxe	Build Now	FAILURE	testes verdes SyntaxError no build; teste pulado
3	...testes- falhando	Build Now	UNSTABLE	build verde; 1 teste vermelho; bola amarela
4	...build- agendado	cron (sozinho)	SUCCESS	"Started by timer"
Bônus	qualquer sucesso	Build Now	SUCCESS	tabela de cobertura + lcov.info